

CARRERA	NOMBRE DE LA ASIGNATURA
ING. SISTEMAS COMPUTACIONALES	LENGUAJES DE INTERFAZ

DOCENTE DESIGNADO

ISC. RICARDO GONZÁLEZ GONZÁLEZ

EQUIPO / INTEGRANTES

GONZÁLEZ PEREZ RODRIGO
MANDUJANO ÁVILA OMAR
LÓPEZ YÁÑEZ LUIS JESÚS
VÁZQUEZ PÉREZ EMILIO FRANCISCO

PRACTICA No.	NOMBRE DE LA PRACTICA	DURACION(HORAS)
10	PROYECTO FINAL	300 HRS

1 INTRODUCCIÓN

El presente proyecto final, desarrollado en el marco de la asignatura Lenguajes de Interfaz, se centra en la concepción e implementación de un brazo robótico antropomórfico con capacidad de teleoperación gestual. Este sistema representa una convergencia multidisciplinaria entre la robótica, la visión artificial (Computer Vision) y la programación de bajo nivel para el control de hardware, abordando de manera práctica los principios fundamentales de las interfaces hombre-máquina (HMI) y la cinemática robótica.

Fundamentación y Contexto

La interacción natural y el control intuitivo de sistemas robóticos son áreas críticas en la automatización industrial, la medicina (telecirugía) y la asistencia personal. Tradicionalmente, la programación de brazos robóticos ha dependido de interfaces textuales complejas o de la memorización de trayectorias (teach-pendant). Este proyecto propone un paradigma de control más directo e inmersivo, donde la barrera entre el operador y el robot se minimiza.

El sistema implementado utiliza un brazo robótico articulado impulsado por una configuración de servomotores. El control de estos actuadores se realiza a través de un *script* desarrollado en Python, elegido por su robustez, su amplio ecosistema de librerías para el procesamiento de imágenes, y su facilidad para la interconexión con dispositivos de *hardware* a nivel de interfaz.

La documentación subsiguiente detallará la arquitectura del *hardware* (componentes electrónicos y mecánicos), el desarrollo del *software* de visión artificial y la lógica de control en Python, y la descripción del protocolo de comunicación serial implementado, proporcionando una visión completa de este ambicioso proyecto de integración percepción-acción en un sistema robótico.

2	OBJETIVOS (COMPETENCIAS)
	Objetivo General Diseñar e implementar un Sistema Integrado de Control Robótico (S.I.C.R.) capaz de realizar la teleoperación de un brazo articulado de múltiples grados de libertad. Este control se efectuará en tiempo real mediante la detección y el seguimiento de la pose de una mano humana a través de algoritmos de visión artificial programados en Python, demostrando la aplicación práctica de los conocimientos de la asignatura Lenguajes de Interfaz. Objetivos Específicos Para lograr el objetivo general, se establecerán los siguientes puntos clave: 1. Componente de Hardware y Mecánica (Plataforma Física) - Diseñar y ensamblar la estructura física del brazo robótico, asegurando una configuración que permita los grados de libertad necesarios para replicar los movimientos fundamentales de la mano y muñeca humanas. - Integrar y calibrar adecuadamente los servomotores necesarios en cada articulación, verificando su correcto funcionamiento y la precisión angular requerida. - Establecer la conexión eléctrica y la interfaz de bajo nivel (microcontrolador) que reciba e interprete los comandos de control provenientes del <i>script</i> de Python. 2. Componente de Visión Artificial (Adquisición y Procesamiento de Datos) - Desarrollar un módulo de visión artificial en Python (utilizando librerías como OpenCV o MediaPipe) que permita la detección fiable y el seguimiento de los puntos clave (landmarks) de la mano humana. - Implementar el algoritmo necesario para la extracción de parámetros cinemáticos a partir de la pose detectada, traduciendo la información visual 2D o 3D a un conjunto de ángulos o coordenadas que definan la posición de la mano del operador. 3. Componente de Software y Control (Lógica de Mapeo) - Crear la lógica de mapeo gestual, que transforme los ángulos o coordenadas de la mano detectada en los comandos angulares de entrada específicos para cada uno de los servomotores del brazo robótico. - Garantizar una comunicación fluida y de baja latencia a través de un protocolo de comunicación serial (o equivalente) entre el <i>script</i> de Python (alto nivel) y el <i>hardware</i> (bajo nivel). 4. Documentación y Divulgación (Página Web Explicativa) - Desarrollar una página web interactiva que sirva como herramienta de divulgación del proyecto. - Explicar de forma clara y accesible los conceptos fundamentales de la robótica, visión artificial y lenguajes de interfaz que sustentan el proyecto. - Argumentar la motivación y justificación del proyecto, resaltando su potencial impacto y las soluciones que aporta en el campo de la teleoperación gestual 5. Validación y Rendimiento (Resultado Final) - Validar la funcionalidad completa del sistema logrando que el brazo robótico replique fielmente y en tiempo real una secuencia de movimientos básicos de la mano, demostrando la sincronización entre el estímulo visual y la respuesta motriz. - Documentar de manera clara y exhaustiva el diseño del <i>software</i>, la configuración del

hardware y los resultados de las pruebas realizadas, sirviendo como manual técnico y memoria del proyecto.

4**MARCO TEÓRICO REFERENCIAL**

Este marco teórico establece los cimientos epistemológicos y tecnológicos de la solución propuesta, la cual se sitúa en la intersección de la Robótica Telerrobótica, la Visión Artificial Estocástica y las Arquitecturas de Comunicación por Protocolos de Bajo Nivel.

1. Cinética Robótica Avanzada y Actuación Precisa

La réplica fiel del movimiento humano requiere una comprensión rigurosa de la cinemática y las características dinámicas de los actuadores.

1.1. Modelado Cinemático y Transformaciones Homogéneas

Para la implementación de la Cinemática Inversa (CI), se recomienda el uso del formalismo de Denavit-Hartenberg (D-H). Este método sistemático permite asignar un sistema de coordenadas a cada articulación del robot, definiendo la relación espacial entre ellas mediante cuatro parámetros: a_i (longitud del eslabón), α_i (torsión del eslabón), d_i (desplazamiento de la articulación) y θ_i (ángulo de la articulación).

La posición y orientación del efector final se calculan a través de la multiplicación secuencial de matrices de transformación homogénea T_i^{i-1} :

$$T_{efector}^{base} = T_1^0(\theta_1, \alpha_1, a_1, d_1) \cdot T_2^1(\theta_2, \alpha_2, a_2, d_2) \cdot \dots \cdot T_n^{n-1}(\theta_n, \alpha_n, a_n, d_n)$$

La CI, fundamental para este proyecto, resuelve n ángulos de articulación ($\theta_1, \dots, \theta_n$) dada una posición (x, y, z) y orientación (R) del efector final. En brazos de alta redundancia, la CI puede tener múltiples soluciones, lo que requiere aplicar criterios de optimización para seleccionar la trayectoria más fluida o la que evite singularidades.

1.2. Control PID en Servomotores

Los servomotores utilizados son intrínsecamente sistemas de control de posición de lazo cerrado. La precisión de la réplica depende de la eficacia de su Controlador Proporcional-Integral-Derivativo (PID) interno.

Proporcional (K_p): Acelera la respuesta del sistema al error, pero puede causar *overshoot* (sobrepaso).

Integral (K_i): Elimina el error en estado estacionario (el robot alcanza exactamente la posición deseada).

Derivativo (K_d): Amortigua la oscilación y mejora la estabilidad, crucial para movimientos rápidos detectados por la visión artificial.

La Modulación por Ancho de Pulso (PWM) es el mecanismo físico utilizado por el microcontrolador para comunicar el ángulo deseado al servo, donde el ancho del pulso

(típicamente entre 1 y 2 ms en un ciclo de 20 ms) determina la posición angular (0° a 180°).

2. Visión Artificial Estocástica y Mapeo Gestual

La percepción del sistema se basa en modelos de Visión Profunda (Deep Learning) para la estimación de la pose humana, lo que introduce un componente estocástico (probabilístico) en la entrada de datos.

2.1. Estimación de la Pose 3D (Z-Depth)

Las soluciones modernas como MediaPipe (o equivalentes) no solo proporcionan coordenadas x e y en el plano de la imagen, sino que también estiman una profundidad relativa z. Esta información es crítica:

Coordenadas Normalizadas: Los landmarks se entregan típicamente en coordenadas normalizadas $[0, 1]$, que deben ser desnormalizadas a la escala real del entorno y, crucialmente, proyectadas al sistema de coordenadas tridimensional del brazo robótico.

Robustez a Oclusiones: Los modelos de Deep Learning ofrecen robustez al inferir la posición de landmarks parcialmente ocluidos (ocultos), una ventaja significativa sobre los métodos tradicionales de procesamiento de imágenes.

2.2. Algoritmo de Mapeo Basado en Cinemática del Efector

El Mapeo Gestual debe ir más allá de una simple asignación de ángulo a ángulo. Se requiere un mapeo basado en las coordenadas cartesianas del efector final de la mano humana:

Detección: Determinar la posición $P_{mano} = (x_h, y_h, z_h)$ y la orientación O_{mano} (dada por la normal de la palma) del punto central de la mano humana.

Transformación y Escalado: Aplicar una transformación lineal y un factor de escalado (homotecia) para ajustar la P_{mano} al espacio de trabajo del robot (W_{robot}). Esto asegura que un pequeño movimiento de la mano no cause un movimiento excesivo y potencialmente dañino en el robot.

$$P_{robot} = k \cdot (P_{mano} - P_{offset})$$

CI para Control: Utilizar la Cinemática Inversa del brazo robótico sobre la P_{robot} calculada para obtener los ángulos de los servomotores.

2.3. Control de la Pinza (Gripper)

El control del efector final (pinza) se puede ligar al cierre de los dedos (e.g., distancia euclidiana entre la punta de los dedos índice y pulgar). Cuando esta distancia cae por debajo de un umbral δ , el software envía el comando para cerrar la pinza, proporcionando un control intuitivo de agarre.

3. Protocolos de Interfaz y Arquitectura Asíncrona

La fluidez de la teleoperación depende de una comunicación eficiente y de alta velocidad entre los dos subsistemas de diferentes niveles de abstracción.

3.1. Serialización de Datos Binarios (En lugar de ASCII)

Para maximizar la eficiencia y reducir la latencia, se recomienda enviar los comandos de ángulo no como cadenas de texto (ASCII), sino como datos binarios serializados (e.g., struct empaquetado).

Ventaja: Un valor angular (e.g., 175) enviado como ASCII ocupa 3 bytes, mientras que como entero de 8 bits sin signo (uint8_t) ocupa solo 1 byte. Esto reduce la carga del bus de comunicación y acelera la transmisión de los N ángulos (e.g., 5-DOF x 1 byte/ángulo = 5 bytes por paquete).

Protocolo: Definir un protocolo de paquete estricto, incluyendo un Byte de Inicio (S_B) y un Byte de Parada (E_B) para la sincronización y validación del paquete, previniendo errores de lectura en el microcontrolador.

3.2. Programación Asíncrona en Python

Para evitar que el script de visión se detenga mientras espera la transmisión serial, se puede emplear la programación asíncrona o threading.

Un **Thread 1** se dedica exclusivamente a la captura de video, procesamiento de imagen y el cálculo de la CI.

Un **Thread 2** (o tarea asíncrona) se encarga de la comunicación serial, enviando los comandos angulares calculados de forma continua, asegurando que el stream de control nunca se detenga.

3.3. Interfaz de Usuario y Configuración (Web-Based)

La página web no solo es de divulgación; puede funcionar como una Interfaz de Configuración (WebUI). Usando marcos como Flask o Django en Python, se pueden implementar funciones para:

Calibración en Línea: Permitir al usuario ajustar los offsets (P_{offset}) o los factores de escalado (k) del mapeo gestual a través de la interfaz web, enviando estos nuevos parámetros al script de control en tiempo real.

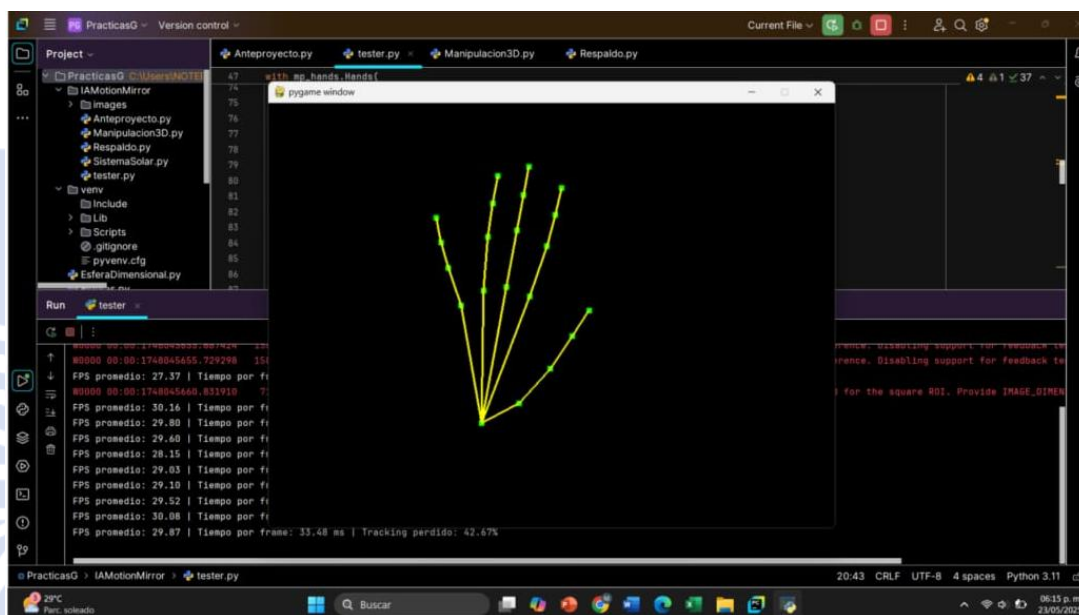
Visualización de Estado: Mostrar la posición actual de los servomotores y la latencia del sistema, proporcionando una herramienta de diagnóstico crucial.

4	MATERIALES UTILIZADOS
	• 4 servo motor MG995 • 1 servo motor MG95 • 1 Arduino • Cables macho macho • Cables hembra macho • Barras de silicón • Pistola de silicón • Resorte • Hilo cáñamo • Pintura acrílica gris • Pintura acrílica azul • Filamento para impresora 3D • Impresora 3D • Lija 200 • Tabla de madera MDF • Segueta para madera • Tornillos • Desarmador de cruz • Pinzas de corte • Pinzas de punta • Cúter • Protoboard • Fuente de alimentación • Cable para el Arduino • Encendedor • Impresora • Papel fotográfico • Pincel • Tijeras • Plumón • Regla • Nivel • Laptop • Python • Pistola de silicón • HTML • Javascript • Css • Netify • Tinkercad

5

DESARROLLO

Los orígenes de esta iniciativa técnica se remontan a **febrero de 2025**, en el marco académico de la asignatura de **Graficación**. En dicha etapa inicial, se planteó la concepción de un sistema avanzado de visión artificial y procesamiento de imágenes en tiempo real. El objetivo primario consistía en desarrollar un algoritmo basado en inteligencia artificial capaz de detectar, segmentar y realizar el seguimiento de una mano humana. Dicho software tenía la función de renderizar en pantalla una representación esquelética digital que replicara, con alta fidelidad y de manera síncrona, cada uno de los movimientos y gestos realizados por el usuario.

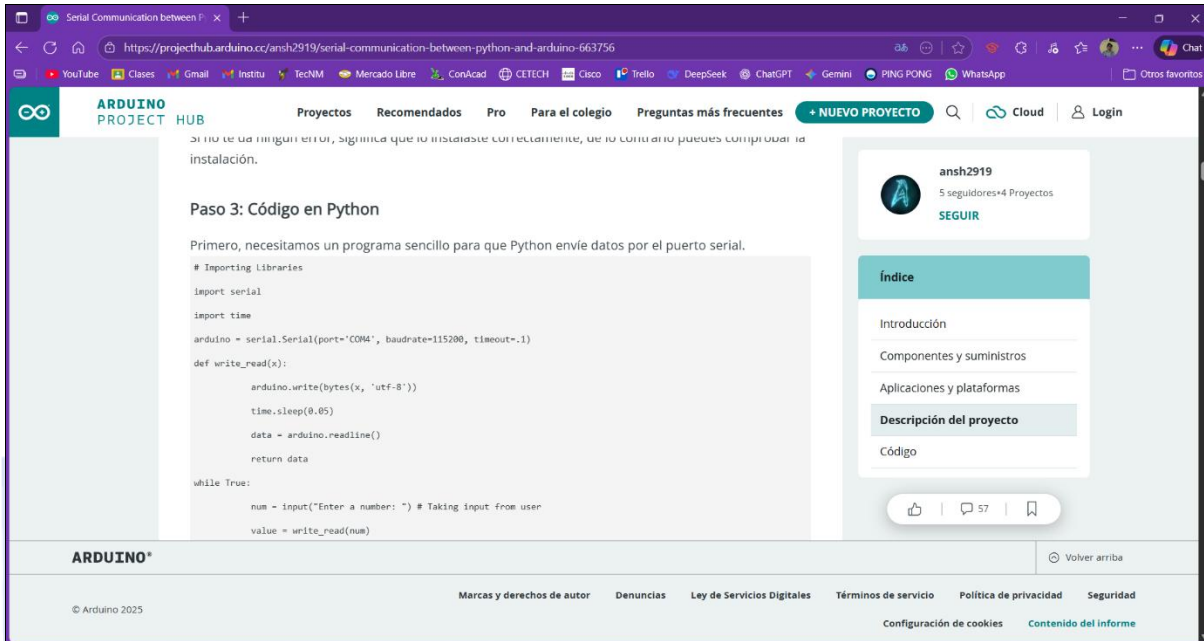


[Imagen del proyecto de Graficación]

Tras la culminación exitosa del proyecto curricular en la materia de Graficación, el equipo de desarrollo evaluó el potencial de la tecnología creada. Fue en este punto donde surgió una propuesta evolutiva ambiciosa: trascender la simulación digital para llevar la aplicación al plano físico. Se estableció como meta la implementación de este software de reconocimiento gestual en un prototipo de prótesis robótica, logrando así que un dispositivo electromecánico replicara físicamente las instrucciones interpretadas por el sistema de procesamiento de imágenes.

Posteriormente, al llegar a **septiembre de 2025**, y situándonos en el contexto de la asignatura de **Lenguajes de Interfaz**, se presentó la oportunidad idónea para materializar esta evolución. Durante las sesiones de lluvia de ideas para definir el proyecto semestral, la propuesta de integración hardware-software fue presentada ante el cuerpo docente. Tras recibir la aprobación y validación académica, se dio inicio formal a la segunda fase del desarrollo.

La primera etapa consistió en una rigurosa fase de investigación y viabilidad técnica. Dado que el núcleo de inteligencia artificial estaba desarrollado en el lenguaje de alto nivel **Python**, y la manipulación de los actuadores electrónicos se realizaría mediante la plataforma de desarrollo **Arduino**, era imperativo establecer un puente de comunicación eficiente. El desafío principal radicaba en validar la interoperabilidad entre ambas tecnologías, consideradas pilares fundamentales para la arquitectura del sistema.

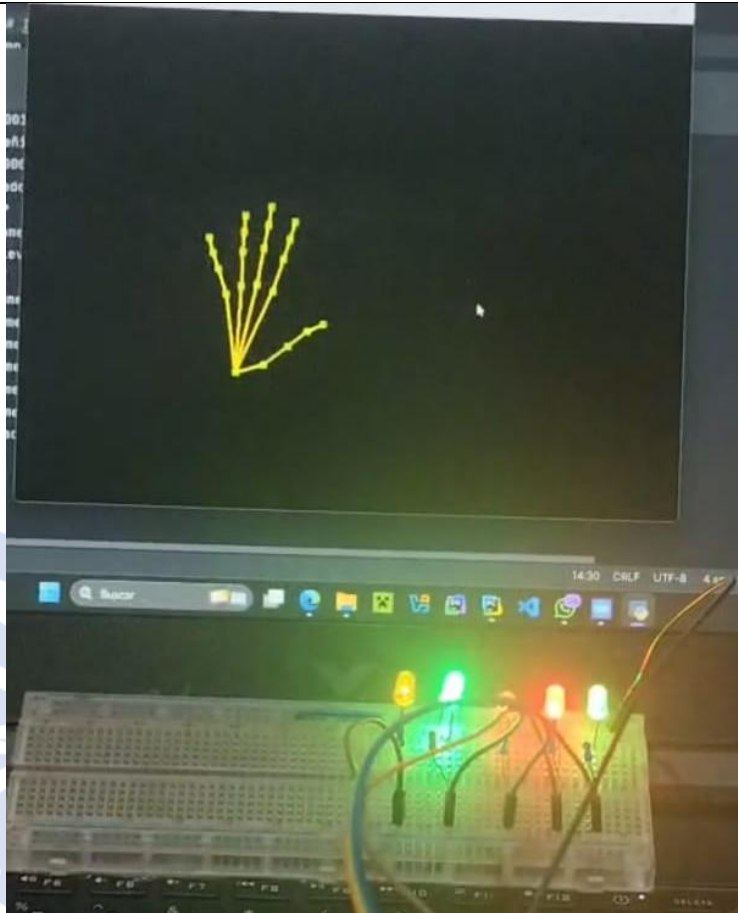


[Imagen de la investigación en foros de la web]

Tras un exhaustivo análisis bibliográfico y técnico, se logró diseñar una solución robusta para la comunicación bidireccional entre el software y el hardware. Se determinó que la metodología óptima consistía en la transmisión de datos a través de comunicación serial utilizando los puertos COM del microcontrolador.

Para optimizar el rendimiento y reducir la latencia, se diseñó un protocolo en el que Python procesa la carga pesada de visión computacional y envía únicamente señales lógicas simplificadas (banderas o valores booleanos) al Arduino. De esta manera, el microcontrolador se dedica exclusivamente a la ejecución de instrucciones de hardware, compensando sus limitaciones de procesamiento frente a la potencia del ordenador.

Con la arquitectura de comunicación definida, se procedió a la construcción de un **prototipo de validación inicial**. Este circuito de prueba tenía como finalidad verificar la lógica de control: cada vez que el sistema de visión artificial detectaba la flexión de un dedo específico, se enviaba una señal para energizar un diodo emisor de luz (LED) correspondiente. Si el dedo permanecía extendido, el LED se mantenía apagado; al flexionarse, el LED se iluminaba, confirmando la correcta interpretación y transmisión de la señal.

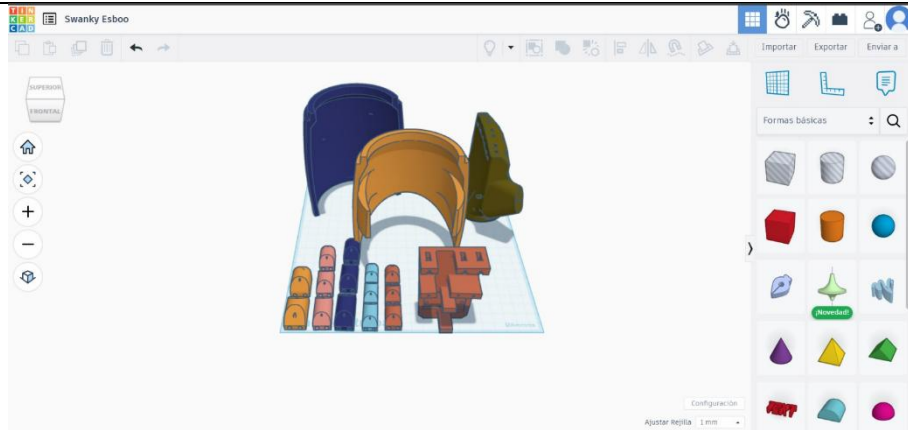


[Imagen de prototipo de leds]

Una vez validada la lógica mediante indicadores luminosos, se avanzó hacia un **prototipo de segunda generación**. En esta fase, se sustituyeron los LEDs por servomotores, dispositivos capaces de realizar movimientos angulares precisos basados en estados definidos.

Durante esta etapa, el equipo enfrentó un desafío técnico significativo relacionado con el consumo energético: la demanda de corriente de los múltiples servomotores excedía la capacidad del puerto USB. Para solventar este impedimento y garantizar la estabilidad del sistema, se implementó una fuente de alimentación externa independiente, logrando así el funcionamiento fluido y simultáneo de todos los actuadores.

Con la electrónica y el código funcionando en armonía, el proyecto entró en la fase de **diseño mecánico y estructural**. Utilizando herramientas de modelado asistido por computadora (CAD), específicamente el software Tinkercad, se diseñó pieza por pieza la estructura anatómica del brazo robótico. Este proceso requirió un enfoque meticuloso para asegurar que las dimensiones y articulaciones emularan correctamente la biomecánica de una mano humana.

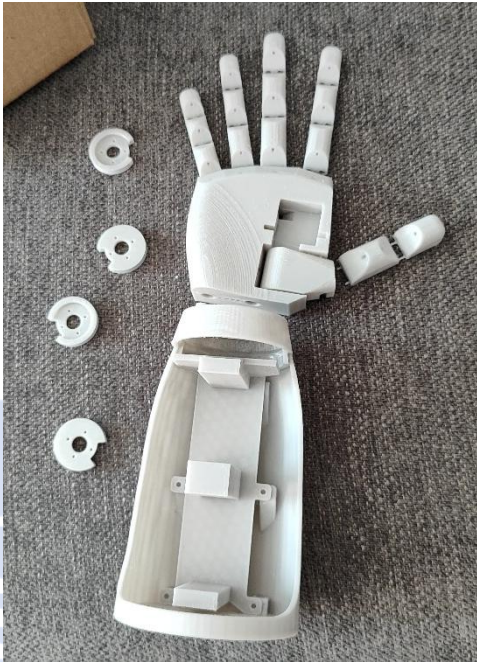


[Imágenes del modelo 3D]

Concluido el modelado digital, se procedió a la manufactura aditiva mediante impresión 3D. Este proceso resultó ser una labor extensa y compleja, ya que, debido a las limitaciones del volumen de impresión, fue necesario fabricar el brazo por secciones independientes.

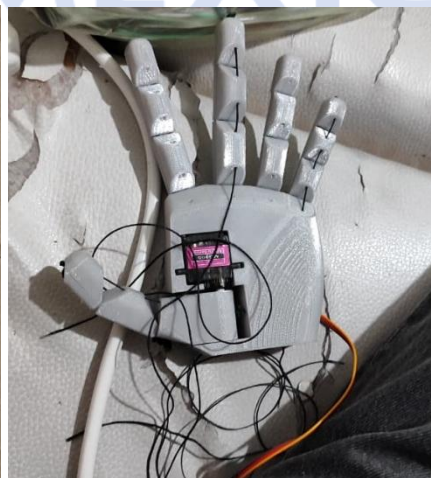
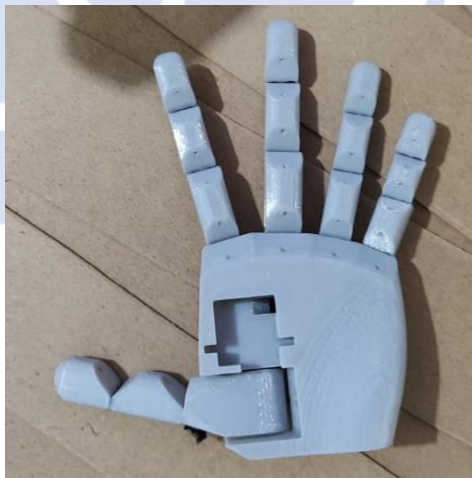
Al finalizar la impresión de todos los componentes, se inició el ensamblaje estructural. Para dotar al brazo de una capacidad de retorno y movimiento natural, se integraron resortes elásticos de material textil, los cuales cumplen la función biomimética de los tendones, permitiendo la extensión pasiva de los dedos tras la actuación de los motores.





[Imágenes del modelo imprimiéndose]

Continuando con la integración electromecánica, se realizó la conexión cinemática entre los servomotores y las falanges de los dedos robóticos. Para ello, se utilizó hilo de alta resistencia, enhebrado cuidadosamente a través de canales internos diseñados específicamente en la estructura impresa, transformando así el torque de los motores en movimiento lineal de tracción.





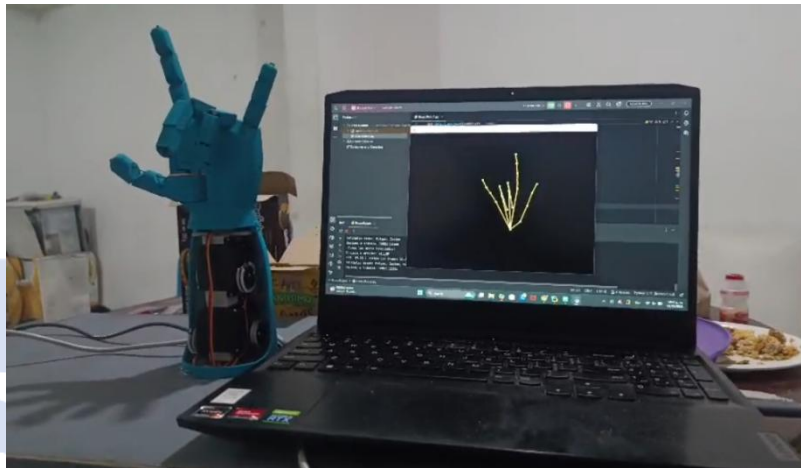
[Imágenes sobre el ensamblado de los hilos]

Paralelamente, se abordó el diseño del chasis y soporte del sistema. Se consideró esencial contar con una base que no solo brindara estabilidad mecánica, sino que también permitiera ocultar el cableado para ofrecer una estética limpia y profesional. Siguiendo principios de reutilización y sostenibilidad, se empleó madera MDF remanente de un proyecto anterior. Se cortaron y ensamblaron los paneles siguiendo medidas específicas de **60x40 cm**, creando una estación de trabajo robusta para el brazo robótico.



[Imágenes de la base armando]

Finalmente, se procedió a la integración total de los subsistemas: el anclaje del brazo robótico a la base, la conexión final de la electrónica y la sincronización con el software de control. Con el ensamblaje completado y las pruebas de calibración finalizadas, el proyecto quedó listo para su presentación y defensa académica, demostrando la exitosa convergencia entre la visión artificial y la robótica.



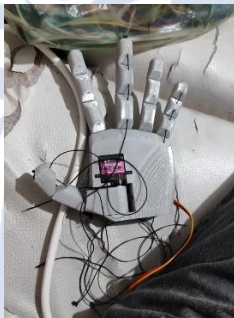
[Imágenes del proyecto acabado]

6	BITÁCORA DE INCIDENCIAS
	➤ 12 de noviembre del 2025. Al tratar de realizar el prototipo de la conexión de los servomotores nos dimos cuenta de que los servomotores una fuente de alimentación externa, por lo que nos afectó en el proceso del prototipo, por lo que tuvimos que utilizar la laptop como la fuente de alimentación. ➤ 15 de noviembre. En el proceso del diseño de las piezas, para ensamblar el brazo, tuvimos problemas con las medidas en el diseño que se estaba realizando con Tinkercad lo que nos atrasó para poder ir armando el programa, para la solución volvimos hacer las medidas y volvimos a hacer los diseños. ➤ 18 de noviembre. Al momento de probar el código, que detecta el movimiento de la mano con IA, se presentaron problemas con las librerías al momento de ejecutar el programa en otro equipo de cómputo, lo que se tuvo que hacer fue instalar una versión más reciente del pycharm, para que pudiera leer todas las librerías. ➤ 25 de noviembre. En el proceso de la impresión se nos acabo el filamento para la impresora 3d por lo que tuvimos que encargar y nos retraso en la impresión de las piezas.

- 27 de noviembre. En el proceso de pegar la base de la maqueta, tuvimos problemas con las medidas ya que la tapa de la base donde se guarda el circuito no entraba de manera correcta, lo que tuvimos que volver a pegar la tapa y lijarla, para alcanzar el tamaño exacto de la tapa.



- 30 de noviembre. En el proceso de ensamblado del brazo robótico, tuvimos problemas con el espacio donde van los hilos, tuvimos que cambiar el grosor del hilo, para que pudiera entrar de manera correcta y pudiera hacer los movimientos sin problemas.



7	OBSERVACIONES
	➤

8	ANEXOS
	➤ VIDEO https://drive.google.com/file/d/1Mfneg9hOZD1MylmS2vqjDK8QVMy2qO-K/view?usp=drive_link

--

9	REFERENCIAS
	✓ Guía de soluciones de MediaPipe. (s. f.). Google AI For Developers. https://ai.google.dev/edge/mediapipe/solutions/guide?hl=es-419 ✓ Opencv. (s. f.). GitHub - opencv/opencv: Open Source Computer Vision Library. GitHub. https://github.com/opencv/opencv ✓ Teknic, Inc. (2021, 5 mayo). Servo Comparison Teknic, Inc. https://teknic.com/servo-comparison/?gad_source=1&gad_campaignid=12487070597&gbraid=0AAAAAD-w-qBICrUH5M32O1Y7bPPz79wLh&gclid=CjwKCAiA3L_JBhAlEiwAlcWO5oSqYDvlZcAxse4vgTr7YfxntWNN6f166u-w-qjegFSvBejSBvtGohoCuNAQAvD_BwE ✓ Welcome to Python.org. (2025, 2 diciembre). Python.org. https://www.python.org.translate.google/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc ✓ De AdvancedMotionControls, P. (2025, 30 mayo). What is a Servo Motor: Definition, Origins, Components, Types & Applications. ADVANCED Controles de Movimiento. https://www.a-m-c.com/es/servomotor/

